

```

/*
 * email_priority_heap.c
 * -----
 * Prioritizes a CEO's incoming emails using a MAX HEAP that is built
 * entirely from scratch on top of a dynamic array (a "list"), with no
 * pre-built heap/priority-queue library used.
 *
 * PRIORITY MODEL (documented assumption, since the spec didn't pin
 * one down):
 * 1. Each sender has a rank (see rank_table[]):
 *     Boss / CEO      -> 100 (highest)
 *     ImportantPerson -> 80
 *     Peer            -> 50
 *     Subordinate     -> 30
 *     OtherPerson     -> 10
 *     <unknown sender> -> 20 (default)
 * 2. Tie-break: if two emails have the same sender rank, the one
 *     with the MORE RECENT date is considered higher priority (the
 *     newest message from that person is the freshest/most likely
 *     to still be relevant, e.g. a later "Never Mind" outranks an
 *     earlier "Important" from the same sender).
 *
 * COMMANDS (read one per line from stdin):
 * EMAIL sender,subject,date -> parse and insert into the heap
 * COUNT                     -> print how many emails are queued
 * NEXT                      -> remove the top-priority email from
 *                           the heap and hold it as "current"
 *                           (no console output by itself)
 * READ                     -> print the "current" email (the one
 *                           most recently pulled off by NEXT)
 *
 * Build:  gcc -Wall -o email_priority_heap email_priority_heap.c
 * Run (pass the file directly):
 *     ./email_priority_heap sample_input.txt
 * Run (via shell redirection, still supported):
 *     ./email_priority_heap < sample_input.txt
 * Run (type commands interactively, still supported):
 *     ./email_priority_heap
 */

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
#include <strings.h> /* strcasecmp / strncasecmp */

#define MAX_FIELD 256

/* ----- */
/* Email record
 */

```

```

/* ----- */

typedef struct {
    char sender[MAX_FIELD];
    char subject[MAX_FIELD];
    char date[MAX_FIELD]; /* stored as given, format MM-DD-YYYY */
    int priority;          /* numeric rank derived from sender */
} Email;

/* ----- */
/* Max-heap: a plain dynamic array ("list") plus size/capacity */
/* ----- */

typedef struct {
    Email *data; /* contiguous array holding the heap */
    int size; /* number of emails currently stored */
    int capacity; /* allocated slots in data[] */
} MaxHeap;

static void heap_init(MaxHeap *h) {
    h->capacity = 8;
    h->size = 0;
    h->data = malloc(sizeof(Email) * (size_t)h->capacity);
    if (!h->data) {
        fprintf(stderr, "Out of memory\n");
        exit(1);
    }
}

static void heap_grow(MaxHeap *h) {
    h->capacity *= 2;
    Email *tmp = realloc(h->data, sizeof(Email) * (size_t)h->capacity);
    if (!tmp) {
        fprintf(stderr, "Out of memory\n");
        exit(1);
    }
    h->data = tmp;
}

static void heap_free(MaxHeap *h) {
    free(h->data);
    h->data = NULL;
    h->size = h->capacity = 0;
}

static int idx_parent(int i) { return (i - 1) / 2; }
static int idx_left(int i) { return 2 * i + 1; }
static int idx_right(int i) { return 2 * i + 2; }

static void swap_email(Email *a, Email *b) {

```

```

    Email t = *a;
    *a = *b;
    *b = t;
}

/* Converts "MM-DD-YYYY" into an integer YYYYMMDD so dates compare
 * correctly with plain integer comparison. Returns 0 on bad input. */
static int date_key(const char *date) {
    int mm = 0, dd = 0, yyyy = 0;
    if (sscanf(date, "%d-%d-%d", &mm, &dd, &yyyy) != 3) return 0;
    return yyyy * 10000 + mm * 100 + dd;
}

/* Returns 1 if email a outranks email b in the max-heap ordering. */
static int higher_priority(const Email *a, const Email *b) {
    if (a->priority != b->priority)
        return a->priority > b->priority;
    /* Same rank: the more recent (later) date wins the tie */
    return date_key(a->date) > date_key(b->date);
}

static void sift_up(MaxHeap *h, int i) {
    while (i > 0 && higher_priority(&h->data[i], &h->data[idx_parent(i)])) {
        swap_email(&h->data[i], &h->data[idx_parent(i)]);
        i = idx_parent(i);
    }
}

static void sift_down(MaxHeap *h, int i) {
    for (;;) {
        int l = idx_left(i), r = idx_right(i), best = i;
        if (l < h->size && higher_priority(&h->data[l], &h->data[best])) best = l;
        if (r < h->size && higher_priority(&h->data[r], &h->data[best])) best = r;
        if (best == i) break;
        swap_email(&h->data[i], &h->data[best]);
        i = best;
    }
}

static void heap_insert(MaxHeap *h, Email e) {
    if (h->size == h->capacity) heap_grow(h);
    h->data[h->size] = e;
    sift_up(h, h->size);
    h->size++;
}

/* Removes and returns the highest-priority email. Returns 0 if empty. */
static int heap_extract_max(MaxHeap *h, Email *out) {
    if (h->size == 0) return 0;
    *out = h->data[0];

```

```

    h->size--;
    h->data[0] = h->data[h->size];
    sift_down(h, 0);
    return 1;
}

/* ----- */
/* Sender -> priority rank lookup */
/* ----- */

typedef struct {
    const char *name;
    int rank;
} SenderRank;

static const SenderRank rank_table[] = {
    { "Boss",          100 },
    { "CEO",           100 },
    { "ImportantPerson", 80 },
    { "Peer",          50 },
    { "Subordinate",   30 },
    { "OtherPerson",   10 },
};

#define RANK_TABLE_SIZE (int)(sizeof(rank_table) / sizeof(rank_table[0]))

static int sender_priority(const char *sender) {
    for (int i = 0; i < RANK_TABLE_SIZE; i++) {
        if (strcmp(sender, rank_table[i].name) == 0)
            return rank_table[i].rank;
    }
    return 20; /* default rank for a sender not in the table */
}

/* ----- */
/* Parsing helpers */
/* ----- */

/* Strips a trailing \n / \r (from fgets) AND any trailing spaces or
 * tabs (easy to pick up when hand-typing or pasting commands), so
 * "COUNT ", "COUNT\r\n" and "COUNT" all match the same command. */
static void trim_line(char *s) {
    size_t len = strlen(s);
    while (len > 0 &&
        (s[len - 1] == '\n' || s[len - 1] == '\r' ||
         s[len - 1] == ' ' || s[len - 1] == '\t')) {
        s[--len] = '\0';
    }
}

/* Parses "sender,subject,date" (the text after "EMAIL ").

```

```

* Splits on the FIRST comma (end of sender) and the LAST comma
* (start of date); everything between is the subject, so a subject
* containing commas is still handled correctly. */
static int parse_email(const char *text, Email *e) {
    const char *first_comma = strchr(text, ',');
    if (!first_comma) return 0;
    const char *last_comma = strrchr(text, ',');
    if (!last_comma || last_comma == first_comma) return 0;

    size_t sender_len = (size_t)(first_comma - text);
    size_t subject_len = (size_t)(last_comma - first_comma - 1);
    size_t date_len = strlen(last_comma + 1);

    if (sender_len >= MAX_FIELD || subject_len >= MAX_FIELD || date_len >=
MAX_FIELD)
        return 0;

    strncpy(e->sender, text, sender_len);
    e->sender[sender_len] = '\0';

    strncpy(e->subject, first_comma + 1, subject_len);
    e->subject[subject_len] = '\0';

    strncpy(e->date, last_comma + 1, date_len);
    e->date[date_len] = '\0';

    e->priority = sender_priority(e->sender);
    return 1;
}

/* Prints the email in the exact block format the user wants to see:
*   Next email:
*   Sender: <sender>
*   Subject: <subject>
*   Date: <date>
*/
static void print_email(const Email *e) {
    printf("Next email:\n");
    printf("Sender: %s\n", e->sender);
    printf("Subject: %s\n", e->subject);
    printf("Date: %s\n", e->date);
}

static void print_count(int size) {
    if (size == 1)
        printf("There is 1 email to read.\n");
    else
        printf("There are %d emails to read.\n", size);
}

```

```

/* ----- */
/* Main: reads commands from stdin, one per line */
/* ----- */

int main(int argc, char *argv[]) {
    MaxHeap heap;
    heap_init(&heap);

    Email current; /* holds the last email pulled off by NEXT */
    int has_current = 0;

    /* Read from a file named on the command line if one was given,
     * e.g. ./email_priority_heap sample_input.txt
     * Otherwise fall back to stdin, e.g. ./email_priority_heap < sample_input.txt
     * or typing commands directly into the running program. */
    FILE *in = stdin;
    if (argc > 1) {
        in = fopen(argv[1], "r");
        if (!in) {
            fprintf(stderr, "Could not open file: %s\n", argv[1]);
            heap_free(&heap);
            return 1;
        }
    }

    char line[1024];

    while (fgets(line, sizeof(line), in)) {
        trim_line(line);
        if (line[0] == '\0') continue;

        if (strncasecmp(line, "EMAIL ", 6) == 0) {
            Email e;
            if (parse_email(line + 6, &e)) {
                heap_insert(&heap, e);
            } else {
                fprintf(stderr, "Malformed EMAIL line: %s\n", line);
            }
        } else if (strcasecmp(line, "COUNT") == 0) {
            print_count(heap.size);
        } else if (strcasecmp(line, "NEXT") == 0) {
            /* Dequeue the highest-priority email and hold onto it;
             * printing happens on the following READ command. */
            if (heap_extract_max(&heap, &current)) {
                has_current = 1;
            } else {
                has_current = 0;
                printf("No more emails.\n");
            }
        } else if (strcasecmp(line, "READ") == 0) {

```

```
        if (has_current) {
            print_email(&current);
        } else {
            printf("No email to read yet - call NEXT first.\n");
        }
    } else {
        fprintf(stderr, "Unknown command: %s\n", line);
    }
}

if (in != stdin) fclose(in);
heap_free(&heap);
return 0;
}
```